LECTURE 7

Reinforcement Learning Part 2: Policy Gradient Methods

Reference Text

REINFORCE, baselines and advantages, actor–critic, PPO, Gaussian policies for continuous control, soft actor–critic — applied to a four-drone swarm

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 Why Optimise the Policy Directly? 3

2 The Policy Gradient Theorem 3

3 REINFORCE 4

3.1 Algorithm 4

3.2 The variance problem 4

4 Baselines and Advantages 4

5 Actor–Critic Methods 4

6 Proximal Policy Optimisation 5

6.1 Why step size matters 5

6.2 The clipped objective 5

7 Continuous Actions: the Gaussian Policy 6

8 Soft Actor–Critic 6

9 Results, Robustness and the Sim-to-Real Gap 6

10 Choosing a Method 7

11 Deeper Dive I: Seeing the Variance Reduction 7

12 Deeper Dive II: Generalised Advantage Estimation 8

13 Deeper Dive III: The Complete PPO Algorithm 8

14 Deeper Dive IV: Bounded Actions and Deterministic Policies 9

14.1 Squashing with tanh 9

14.2 Deterministic policy gradients: DDPG and TD3 9

15 Deeper Dive V: Inside Soft Actor–Critic 9

15.1 Soft Bellman targets 9

15.2 Actor and temperature updates 9

16 Deeper Dive VI: Learning Curves and Robustness 10

17 Deeper Dive VII: One Actor–Critic Update by Hand 10

18 Deeper Dive VIII: Tuning and Troubleshooting 11

Part C — Further Theory, Methods and Applications **12**

19 The Policy Gradient Theorem from First Principles **12**
19.1 Gradient of the expected return over trajectories . . . 12
19.2 Reward-to-go . . . 12

20 Score Functions of Common Policies **12**
20.1 Softmax policy over discrete actions . . . 12
20.2 Gaussian policy over continuous actions . . . 13

21 Natural Gradients, TRPO and Why PPO Works **13**
21.1 Step size in the wrong units . . . 13
21.2 The natural gradient and TRPO . . . 13
21.3 Entropy and its temperature . . . 13

22 Gradient-Free Policy Search **14**

23 Learning From Demonstrations **14**
23.1 Behavioural cloning and compounding error . . . 14
23.2 DAgger . . . 14
23.3 Inverse RL and adversarial imitation . . . 14
23.4 Combining demonstrations with RL . . . 15

24 Hierarchical Control **15**

25 Learned Control Versus Classical Control **15**
25.1 What classical control already does well . . . 15
25.2 Where RL adds value . . . 15
25.3 Model-based RL . . . 15

26 Crossing the Reality Gap **16**

27 Assuring a Learned Controller **16**

28 Designing a Reward for a Drone Swarm **17**

29 Further Self-Test Questions **17**

30 Additional Self-Test Questions **17**

31 Across the Disciplines **18**

32 Self-Test Questions **18**

33 Glossary **18**

1 Why Optimise the Policy Directly?

Value-based methods choose $\arg\max_a Q(s,a)$. This works for a few discrete actions but fails for **continuous** actions — drone thrust in $[-100, 100]^3$ newtons has infinitely many values, so the argmax becomes an optimisation inside every decision — and is awkward for very large discrete action sets. **Policy-gradient** methods parameterise the policy $\pi_\theta(a \mid s)$ itself and adjust θ to increase expected return. For continuous control the policy outputs the parameters of a distribution, typically a Gaussian with mean $\mu_\theta(s)$ and standard deviation $\sigma_\theta(s)$; actions are sampled, so exploration is built in.

Case study. Four drones must visit five waypoints in order while avoiding three no-fly zones. Each drone's state has 18 features (position, velocity, orientation, angular rates, fuel, waypoint offset, safety flags); the joint state adds swarm centroid and formation metrics (80 features). The action is a continuous 3-D thrust per drone (12 dimensions). Simplified dynamics: $v_{t+1} = v_t + (F_t/m - g\hat{z})\Delta t$, $x_{t+1} = x_t + v_{t+1}\Delta t$, with $m = 1.2$ kg and $\Delta t = 0.1$ s. Rewards favour waypoint progress and spacing within 50 m, and penalise spacing under 10 m and collisions.

2 The Policy Gradient Theorem

Derivation — the score-function (log-derivative) trick

Let $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \sum_\tau P(\tau;\theta)R(\tau)$ over trajectories τ. Then

$$\nabla_\theta J = \sum_\tau \nabla_\theta P(\tau;\theta)R(\tau) = \sum_\tau P(\tau;\theta)\nabla_\theta \log P(\tau;\theta)\, R(\tau) = \mathbb{E}\big[\nabla_\theta \log P(\tau;\theta) R(\tau)\big],$$

using $\nabla P = P\nabla \log P$. Since $P(\tau;\theta) = p(s_0)\prod_t \pi_\theta(a_t|s_t)P(s_{t+1}|s_t,a_t)$ and the dynamics do not depend on θ, $\nabla_\theta \log P(\tau;\theta) = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$. Because an action cannot influence rewards received before it, $R(\tau)$ can be replaced by the reward-to-go G_t:

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\Big].$$

Interpretation. $\nabla_\theta \log \pi_\theta(a|s)$ points in the direction that makes action a more likely; multiplying by G_t pushes harder for actions followed by high returns. No model of the environment is needed — only the ability to sample trajectories.

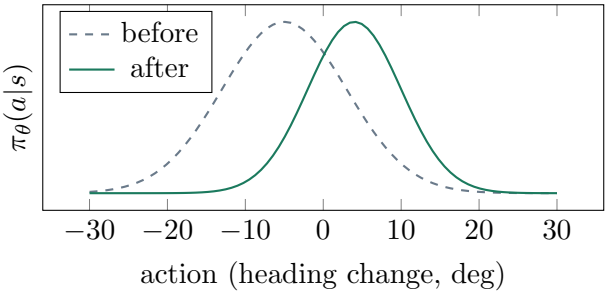


Figure 1: a policy-gradient step shifts probability toward actions that led to high return.

3 REINFORCE

3.1 Algorithm

REINFORCE (Williams, 1992): run the current policy for a complete episode; compute G_t for every step; update $\theta \leftarrow \theta + \alpha \sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$; repeat. It is unbiased and works for discrete and continuous actions, but needs complete episodes, uses each episode once, and has very high variance.

Worked example — one REINFORCE step for a two-action policy

A softmax policy over {left, right} with preferences $h = (0, 0)$ gives $\pi = (0.5, 0.5)$. The agent chose right and the return was $G = 4$. For a softmax, $\nabla_h \log \pi(\text{right}) = \mathbf{1}_{\text{right}} - \pi = (-0.5,\ 0.5)$. With $\alpha = 0.1$: $h \leftarrow (0, 0) + 0.1 \times 4 \times (-0.5, 0.5) = (-0.2,\ 0.2)$, so $\pi(\text{right}) = e^{0.2}/(e^{0.2} + e^{-0.2}) = 0.60$. Had the return been -4, right would have become less likely.

On the drone task, REINFORCE reached 61.4% waypoint success with return 142.8 ± 84.6; it needed 6,200 episodes to reach 50% success and never reached 70%.

3.2 The variance problem

G_t mixes the quality of action a_t with all the luck of the rest of the episode. A poor turn followed by good fortune ($G = 180$) is reinforced; a good turn followed by bad fortune ($G = 40$) is discouraged. Learning still averages out correctly, but needs many samples.

4 Baselines and Advantages

Derivation — a baseline adds no bias

For any function $b(s)$ that does not depend on the action,

$$\mathbb{E}_{a\sim\pi}\left[\nabla_\theta \log \pi_\theta(a|s)\, b(s)\right] = b(s) \sum_a \pi_\theta(a|s) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} = b(s) \nabla_\theta \sum_a \pi_\theta(a|s) = b(s) \nabla_\theta 1 = 0.$$

Hence $\theta \leftarrow \theta + \alpha(G_t - b(s_t)) \nabla \log \pi$ has the same expected update but lower variance when b approximates G_t.

The natural baseline is the state value $V^\pi(s)$, giving the **advantage** $A(s, a) = Q(s, a) - V(s)$: how much better this action was than average from this state. Positive advantage increases the action's probability; negative decreases it. A learned baseline $V_\phi(s)$ raised drone success from 61.4% to 72.1% and halved the return spread ($\pm 84.6 \rightarrow \pm 41.2$).

5 Actor–Critic Methods

An **actor** π_θ chooses actions; a **critic** V_ϕ estimates state values. Instead of waiting for the episode's end, the critic provides a one-step estimate of the advantage, the TD error:

$$\hat{A}_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

This has much lower variance than $G_t - V(s_t)$ but some bias, because V_ϕ is imperfect. Losses: actor $L_\pi = -\mathbb{E}[\hat{A} \log \pi_\theta(a|s)]$; critic $L_V = \mathbb{E}[(G_t - V_\phi(s))^2]$; combined $L = L_\pi + c_1 L_V - c_2 H(\pi_\theta)$, where the entropy bonus H discourages premature convergence to a deterministic policy.

Generalised advantage estimation interpolates between the one-step TD advantage and the Monte-Carlo advantage: $\hat{A}_t^{\mathrm{GAE}} = \sum_{l\geq 0}(\gamma\lambda)^l \delta_{t+l}$ with $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$; $\lambda = 0$ gives low variance and more bias, $\lambda = 1$ the reverse (typically $\lambda = 0.95$).

A2C and A3C. Several workers collect experience in parallel environments. A2C synchronises their updates (diverse data, more stable gradients); A3C lets each worker update a shared network asynchronously (fast on CPUs, but with stale gradients). With four workers A2C reached 83.2% success, return 194.8 ± 28.4.

6 Proximal Policy Optimisation

6.1 Why step size matters

Policy-gradient estimates are only valid near the current policy. A large step can change behaviour drastically — every drone suddenly crashing — and recovery takes thousands of episodes. TRPO (Schulman et al., 2015) constrained each update by a KL-divergence trust region with a monotonic-improvement guarantee, but needs second-order computations.

6.2 The clipped objective

PPO (Schulman et al., 2017) uses the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ and maximises

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\left(r_t\hat{A}_t,\ \mathrm{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\right)\Big], \qquad \varepsilon \approx 0.2.$$

Worked example — when clipping bites

A good action ($\hat{A} = +6.8$) has become 58% more likely ($r = 1.58$). Unclipped gain $1.58 \times 6.8 = 10.76$; clipped $1.2 \times 6.8 = 8.16$; the minimum, 8.16, has zero gradient with respect to further increases — the update stops pushing. For a bad action ($\hat{A} = -3$) with $r = 0.7$: $0.7 \times -3 = -2.1$ vs $0.8 \times -3 = -2.4$; the minimum -2.4 is clipped, so the action is not suppressed further this round.

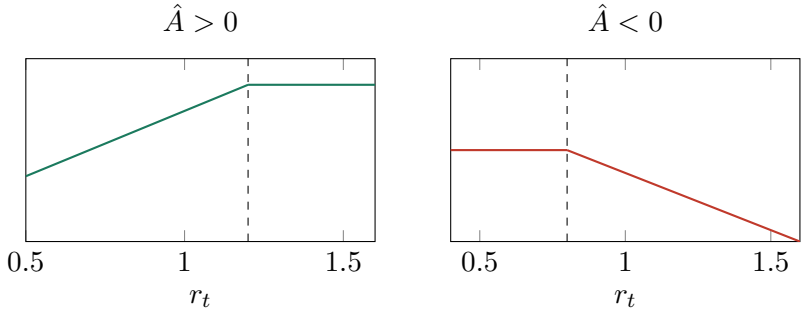


Figure 2: the clipped surrogate. Flat regions give zero gradient, preventing over-large updates.

PPO alternates collecting a batch with the current policy and running several epochs (e.g. 10) of mini-batch SGD on L^{CLIP}. It is simple, robust and the default choice in practice. Drone results: 94.3% success, return spread ± 19.8, 800 episodes to 70% success.

7 Continuous Actions: the Gaussian Policy

$$\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta(s)^2)\big), \qquad \log\pi = -\tfrac{1}{2}\sum_i \Big[\frac{(a_i-\mu_i)^2}{\sigma_i^2} + \log\sigma_i^2 + \log 2\pi\Big].$$

The network outputs μ and $\log\sigma$ (keeping σ positive and training stable). The gradient of $\log\pi$ with respect to the mean is $(a-\mu)/\sigma^2$: if a sampled action above the mean yielded positive advantage, the mean moves up. A state-dependent σ lets the agent explore widely in open airspace and act precisely near obstacles. Actions are usually squashed by tanh into actuator limits, with the corresponding correction to $\log\pi$.

Worked example — a mean update

$\mu = 2.0$, $\sigma = 0.5$, sampled thrust $a = 2.4$, advantage $\hat{A} = +3$. $\partial\log\pi/\partial\mu = (2.4-2.0)/0.25 = 1.6$. With step 0.01: $\mu \leftarrow 2.0 + 0.01 \times 3 \times 1.6 = 2.048$ — toward the action that worked.

8 Soft Actor–Critic

SAC (Haarnoja et al., 2018) maximises reward plus policy entropy:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t\big(r(s_t,a_t) + \alpha H(\pi(\cdot|s_t))\big)\Big], \qquad H = -\mathbb{E}[\log\pi(a|s)].$$

The entropy bonus keeps the policy as random as possible while still achieving high reward. Benefits: automatic exploration; robustness to perturbations (it learns many ways to succeed); unpredictability to adversaries; and good behaviour in multi-modal problems. The temperature α is tuned automatically toward a target entropy. SAC is **off-policy**: it learns from a replay buffer with twin Q-critics (taking the minimum to curb over-estimation) and target networks, making it far more sample-efficient per environment interaction than on-policy PPO.

Drone results. 96.2% waypoint success, collision rate 1.8%, return 218.4 ± 18.7 — the best on every metric. With only spacing rewards and the entropy bonus, the drones spontaneously spread out to cover airspace and converged on waypoints, keeping 32.4 m spacing (between the 10 m penalty and 50 m bonus thresholds): emergent formation flying.

9 Results, Robustness and the Sim-to-Real Gap

Algorithm	Waypoint success	Collisions	Return s.d.	Episodes to 70%
REINFORCE	61.4%	8.4%	±84.6	never (50% at 6,200)
+ baseline	72.1%	6.1%	±41.2	3,800
A2C (4 workers)	83.2%	3.8%	±28.4	2,000
PPO	94.3%	2.4%	±19.8	800
SAC	**96.2%**	**1.8%**	±18.7	1,500

Under ±30% wind perturbation PPO dropped to 71% while SAC held 88%. Field tests revealed the simulation-to-reality gap: PPO fell from 94% in simulation to 78% in real flight because of wind, sensor noise and GPS drift. **Domain randomisation** — training across random wind (±5 m/s), sensor noise (±2 m) and mass and motor variations — raised real-world success to 88% (PPO) and 91% (SAC). If the real world looks like one more random variation of the simulator, the policy transfers.

Pitfall — episodes vs samples

SAC needed more *episodes* than PPO to reach 70% (1,500 vs 800) yet is usually called more sample-efficient, because it reuses every transition many times from replay. Compare algorithms on the resource that is actually scarce: environment interactions, wall-clock compute, or engineering effort.

10 Choosing a Method

Use PPO as the robust default (on-policy, stable, few hyper-parameters). Use SAC for continuous control when environment interaction is expensive and robustness matters. Use DQN-family methods for small discrete action sets where interpretable Q-values are useful for advisory roles. Keep a verified classical controller (PID) as an inner loop and let learned policies set higher-level commands where safety certification matters.

11 Deeper Dive I: Seeing the Variance Reduction

Worked example — why baselines help, numerically

Two episodes from the same state give returns 180 and 40 (the mean is 110). Without a baseline the update weights are 180 and 40; their mean square is $(180^2 + 40^2)/2 = 17{,}000$. With baseline $V(s) = 110$ the weights are $+70$ and -70; mean square 4,900 — about 3.5 times smaller. Both have the same expected gradient, but the second needs far fewer episodes for a reliable estimate, and it sends the right signal (one action better than average, one worse) instead of "both good, one more so".

12 Deeper Dive II: Generalised Advantage Estimation

GAE combines one-step TD errors with exponentially decaying weights:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda\,\hat{A}_{t+1} = \sum_{l\geq 0} (\gamma\lambda)^l \delta_{t+l}.$$

Worked example — a three-step GAE calculation

Rewards $r_1 = 1$, $r_2 = 0$, $r_3 = 2$ (episode ends after r_3); critic values $V(s_0) = 2$, $V(s_1) = 1.5$, $V(s_2) = 1.8$, $V(s_3) = 0$; $\gamma = 0.9$. TD errors: $\delta_0 = 1 + 0.9(1.5) - 2 = 0.35$; $\delta_1 = 0 + 0.9(1.8) - 1.5 = 0.12$; $\delta_2 = 2 + 0 - 1.8 = 0.20$.
With $\lambda = 0.8$ ($\gamma\lambda = 0.72$): $\hat{A}_2 = 0.20$; $\hat{A}_1 = 0.12 + 0.72(0.20) = 0.264$; $\hat{A}_0 = 0.35 + 0.72(0.264) = 0.540$.
With $\lambda = 1$ ($\gamma\lambda = 0.9$): $\hat{A}_1 = 0.30$, $\hat{A}_0 = 0.62$ — exactly the Monte-Carlo advantage $G_0 - V(s_0) = (1 + 0 + 0.81 \times 2) - 2 = 0.62$. With $\lambda = 0$, $\hat{A}_0 = \delta_0 = 0.35$. λ slides between the low-variance (biased) TD estimate and the unbiased (high-variance) Monte-Carlo estimate.

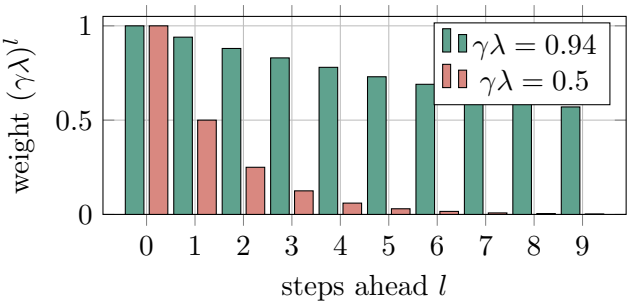


Figure 3: how far ahead GAE looks. Typical PPO settings ($\gamma = 0.99$, $\lambda = 0.95$) give weights decaying slowly.

13 Deeper Dive III: The Complete PPO Algorithm

```
for iteration = 1, 2, ...:
  run policy pi_old in N parallel environments for T steps
  compute advantages A_t with GAE(gamma, lambda) using critic V
  compute value targets R_t = A_t + V(s_t); normalise A_t
  for epoch = 1..K (e.g.  10):
    for each mini-batch:
      ratio = exp(log pi(a|s) - log pi_old(a|s))
      L_clip = mean(min(ratio*A, clip(ratio, 1-eps, 1+eps)*A))
      loss = -L_clip + c1*mean((V(s)-R)^2) - c2*mean(entropy)
      gradient step (Adam, lr about 3e-4), clip gradient norm
    stop early if mean KL(pi_old, pi) > target (e.g.  0.01-0.02)
  pi_old <- pi
```

Typical settings: $\varepsilon = 0.2$, $\gamma = 0.99$, $\lambda = 0.95$, 10 epochs, $c_1 = 0.5$, $c_2 = 0.01$. Monitor the average KL divergence and the fraction of samples clipped: a clip fraction persistently above about 0.3 means the learning rate or number of epochs is too high. Normalising observations and advantages is often as important as the algorithm itself.

14 Deeper Dive IV: Bounded Actions and Deterministic Policies

14.1 Squashing with tanh

Actuators have limits. Sample $u \sim \mathcal{N}(\mu, \sigma^2)$ and output $a = a_{\max} \tanh(u)$. The change of variables adds a correction to the log-probability:

$$\log \pi(a|s) = \log \mathcal{N}(u; \mu, \sigma^2) - \sum_i \log\left(1 - \tanh^2(u_i)\right) \ (+\text{const}).$$

Forgetting this term makes the policy pile probability at the limits.

14.2 Deterministic policy gradients: DDPG and TD3

An alternative is a deterministic actor $a = \mu_\theta(s)$ trained by following the critic's gradient: $\nabla_\theta J = \mathbb{E}[\nabla_a Q(s,a)|_{a=\mu(s)} \nabla_\theta \mu_\theta(s)]$ (DDPG). Exploration noise is added externally. TD3 fixes DDPG's over-estimation with twin critics (taking the minimum), delayed actor updates and target-policy smoothing. SAC can be seen as a stochastic, entropy-regularised successor of these methods; MADDPG (Lecture 8) is the multi-agent extension of DDPG.

15 Deeper Dive V: Inside Soft Actor–Critic

15.1 Soft Bellman targets

SAC learns two critics Q_1, Q_2 and uses the smaller to curb over-estimation. The target includes the entropy term:

$$y = r + \gamma\Big(\min_{j=1,2} Q_j^{\text{targ}}(s', a') - \alpha \log \pi(a'|s')\Big), \qquad a' \sim \pi(\cdot|s').$$

Worked example — a soft target

$r = 1$, $\gamma = 0.99$, target critics give 10.0 and 9.6, the sampled next action has $\log \pi = -1.2$, $\alpha = 0.2$. Then $y = 1 + 0.99(9.6 + 0.24) = 1 + 0.99 \times 9.84 = 10.74$. The entropy term $(+0.24)$ rewards having reached a state where the policy can still act in many ways.

15.2 Actor and temperature updates

The actor minimises $\mathbb{E}[\alpha \log \pi(a|s) - \min_j Q_j(s,a)]$ using reparameterised samples $a = \mu + \sigma\epsilon$ (squashed), so gradients flow through the sample. The temperature is adjusted to hit a target entropy $\bar{H}$ (often $-\dim A$): α rises when the policy is less random than $\bar{H}$ and falls when more. At deployment the mean action is often used, removing stochasticity where predictability matters for certification.

16 Deeper Dive VI: Learning Curves and Robustness

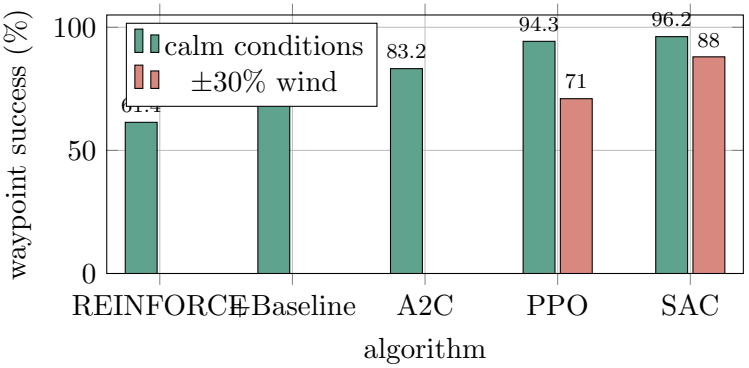


Figure 4: drone-swarm results; SAC's entropy-driven diversity pays off under disturbance.

Domain randomisation should cover every factor that differs between simulator and world: aerodynamics (gusts, ground effect), sensors (noise, latency, dropouts), actuators (motor variation, battery sag), and mass and inertia. Over-randomisation makes the task too hard to learn; a common approach widens the ranges progressively (automatic domain randomisation) as performance improves.

17 Deeper Dive VII: One Actor–Critic Update by Hand

Worked example — a 1-D thrust controller

A drone must hold altitude. Policy $\pi = \mathcal{N}(\mu = w\,e,\ \sigma^2 = 0.25)$ where e is the altitude error (m) and w a single weight; critic $V(e) = -v\,e^2$ with $v = 0.5$. Current $w = 0.8$, $e = 2$ m, so $\mu = 1.6$. The sampled action is $a = 2.0$; the drone moves to $e' = 1$ with reward $r = -1$; $\gamma = 0.9$.

Critic: $V(e) = -2$, $V(e') = -0.5$; TD error $\delta = -1 + 0.9(-0.5) - (-2) = 0.55$ — better than expected.

Actor: $\partial \log \pi / \partial \mu = (a-\mu)/\sigma^2 = (2.0-1.6)/0.25 = 1.6$; $\partial \mu / \partial w = e = 2$; so $\partial \log \pi / \partial w = 3.2$. With actor step 0.01: $w \leftarrow 0.8 + 0.01 \times 0.55 \times 3.2 = 0.8176$ — the policy now responds more strongly to altitude error, because a stronger-than-mean thrust worked.

Critic update: target $-1+0.9(-0.5) = -1.45$ vs prediction -2; with step 0.05 on v: $\partial V/\partial v = -e^2 = -4$; $v \leftarrow 0.5 + 0.05 \times 0.55 \times (-4) = 0.39$, raising $V(e=2)$ to -1.56, closer to the target.

18 Deeper Dive VIII: Tuning and Troubleshooting

Symptom	Likely cause and remedy
Return rises then collapses	Updates too large: lower learning rate, fewer PPO epochs, tighter KL target.
Policy becomes deterministic early	Entropy coefficient too small (PPO) or target entropy too low (SAC): increase it.
Critic loss explodes	Unnormalised rewards or returns: normalise; clip gradients; smaller critic learning rate.
Good in simulation, poor in field	Insufficient domain randomisation; unmodelled latency or sensor noise.
Very slow learning	Sparse rewards: shaping, curriculum, larger λ; check observation normalisation.
High variance across seeds	Report many seeds; use larger batches; consider SAC for sample efficiency.

Key point

Most failures of policy-gradient training are engineering failures — normalisation, step sizes, reward scale — rather than algorithmic ones. Change one thing at a time and log KL divergence, entropy, clip fraction, value loss and episode return.

Part C — Further Theory, Methods and Applications

19 The Policy Gradient Theorem from First Principles

19.1 Gradient of the expected return over trajectories

Let $\tau = (s_0, a_0, r_1, s_1, \ldots)$ be a trajectory and $R(\tau)$ its return. Under policy π_θ,

$$P(\tau;\theta) = p(s_0)\prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

The objective is $J(\theta) = \mathbb{E}_\tau[R(\tau)] = \sum_\tau P(\tau;\theta)R(\tau)$. Using $\nabla P = P\nabla \ln P$ (the log-derivative trick),

$$\nabla J = \sum_\tau P(\tau;\theta)\,\nabla \ln P(\tau;\theta)\, R(\tau) = \mathbb{E}_\tau\Big[R(\tau)\sum_t \nabla \ln \pi_\theta(a_t \mid s_t)\Big],$$

because the initial-state and transition probabilities do not depend on θ and their gradients vanish. The environment's dynamics never need to be known or differentiated: this is why policy gradients are *model-free*.

19.2 Reward-to-go

An action cannot affect rewards received before it was taken, so those rewards only add noise. Replacing $R(\tau)$ by the reward-to-go $G_t = \sum_{k\ge t}\gamma^{k-t} r_{k+1}$ for each term keeps the gradient unbiased and lowers its variance:

$$\nabla J = \mathbb{E}\Big[\sum_t \nabla \ln \pi_\theta(a_t \mid s_t)\, G_t\Big].$$

Subtracting a state-dependent baseline and replacing G_t by an advantage estimate are the further variance reductions developed in the main text.

20 Score Functions of Common Policies

20.1 Softmax policy over discrete actions

With action preferences $h(s,a)$, $\pi(a \mid s) = e^{h(s,a)}/\sum_b e^{h(s,b)}$. The gradient of $\ln \pi(a \mid s)$ with respect to the preferences is the one-hot vector of the chosen action minus the probability vector.

Worked example — one REINFORCE step on preferences

Three manoeuvres (advance, flank, hold) have preferences $(1.0, 0.5, -0.5)$, so $\pi = (0.547, 0.331, 0.122)$. The agent chose *flank* and the return was $G = 4$. Score: $(0,1,0) - \pi = (-0.547, 0.669, -0.122)$. With $\alpha = 0.1$ the preferences move by $\alpha G \times$ score to $(0.781, 0.767, -0.549)$ and the policy becomes $(0.444, 0.438, 0.117)$: flank rose from 33% to 44%, taking probability mainly from advance. Had the return been negative, flank would have been made *less* likely by the same rule.

20.2 Gaussian policy over continuous actions

For $a \sim \mathcal{N}(\mu_\theta(s), \sigma^2)$ with a learned log-standard deviation,

$$\frac{\partial \ln \pi}{\partial \mu} = \frac{a-\mu}{\sigma^2}, \qquad \frac{\partial \ln \pi}{\partial \ln \sigma} = \frac{(a-\mu)^2}{\sigma^2} - 1.$$

Worked example — reading a Gaussian score

A thrust policy has $\mu = 0.50$, $\sigma = 0.20$ and samples $a = 0.62$. Then $\partial \ln \pi / \partial \mu = 0.12/0.04 = 3.0$: with a positive advantage, the mean moves toward the sampled action. And $\partial \ln \pi / \partial \ln \sigma = 0.36 - 1 = -0.64$: because the good action lay within one standard deviation of the mean, a positive advantage *narrows* the distribution; an action that succeeded far out in the tail would widen it. The policy learns both where to aim and how much to explore.

21 Natural Gradients, TRPO and Why PPO Works

21.1 Step size in the wrong units

An ordinary gradient step changes the parameters by a fixed amount, but the same parameter change can alter the policy a little or enormously depending on where it is made. What matters is how much the *behaviour* changes, best measured by the Kullback–Leibler divergence between the old and new action distributions.

Worked example — KL between two thrust policies

For Gaussians, $\mathrm{KL}\big(\mathcal{N}(\mu_1,\sigma_1^2)\,\|\,\mathcal{N}(\mu_2,\sigma_2^2)\big) = \ln\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2+(\mu_1-\mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$. Moving from $\mathcal{N}(0.50, 0.20^2)$ to $\mathcal{N}(0.56, 0.18^2)$ gives KL $= 0.067$: a modest change. Moving the mean to 0.90 with the same spread gives KL $= 2.0$: a drastic change that could crash a drone mid-training. Trust-region methods cap the KL of each update at a value such as 0.01–0.05.

21.2 The natural gradient and TRPO

The **Fisher information matrix** $F = \mathbb{E}[\nabla \ln \pi \, \nabla \ln \pi^\top]$ is the local curvature of the KL divergence. The **natural gradient** $F^{-1}\nabla J$ is the steepest-ascent direction when distance is measured by KL rather than by parameter change. **TRPO** maximises the importance-weighted advantage subject to an average-KL constraint, using conjugate gradients to apply F^{-1} without forming it. It is reliable but complex. **PPO** replaces the hard constraint by the clipped objective, which removes the incentive to push the probability ratio beyond $[1-\varepsilon, 1+\varepsilon]$ and so keeps updates approximately inside a trust region using only first-order optimisation. Monitoring the KL between successive policies remains a useful diagnostic for PPO: a sudden jump signals a learning rate or epoch count that is too large.

21.3 Entropy and its temperature

Adding an entropy bonus $\beta H(\pi(\cdot \mid s))$ to the objective discourages premature collapse onto one action. In SAC the coefficient (temperature α) is tuned automatically to hold the policy's entropy near a target (typically $-\dim(\mathcal{A})$ for continuous actions): the temperature rises when the policy becomes too certain and falls when it is too random.

22 Gradient-Free Policy Search

When the policy has few parameters, or the simulator is a black box run in parallel on many machines, the gradient can be skipped. The **cross-entropy method** (CEM) samples parameters from a distribution, keeps the best-performing fraction (the elite), refits the distribution to the elite and repeats.

Worked example — tuning a controller gain with CEM

Five candidate gains $0.8, 1.2, 1.5, 2.0, 2.6$ are evaluated in simulation with returns $41.9, 47.5, 49.6, 49.1, 41.9$. Keeping the top two (1.5 and 2.0) gives a new sampling distribution with mean 1.75 and standard deviation 0.25, centred near the true optimum of 1.7. Repeating narrows in further.

Evolution strategies perturb all parameters with Gaussian noise and move them in the direction of the return-weighted average perturbation — effectively a gradient estimate by finite differences, easily spread over thousands of processors. These methods are robust to sparse or deceptive rewards and need no back-propagation, but scale poorly to policies with millions of parameters.

23 Learning From Demonstrations

Expert pilots and operators already know how to fly and fight; learning from their demonstrations can give an RL agent a far better starting point than random exploration.

23.1 Behavioural cloning and compounding error

Behavioural cloning treats demonstrations as supervised data: learn $\pi(a \mid s)$ to predict the expert's action. Its weakness is **compounding error**: a small mistake takes the agent into states the expert never visited, where it makes larger mistakes. If the per-step error rate is ε, errors over a horizon of T steps can grow like εT^2 rather than εT; with $\varepsilon = 0.01$ and $T = 100$ that is the difference between about 100 and 1 expected errors in the bound.

23.2 DAgger

Dataset aggregation fixes this by letting the learner fly, asking the expert what they *would* have done in the states the learner actually reached, adding those labels to the data and retraining. The training distribution then matches the states the learner encounters, and the error grows only linearly with the horizon.

23.3 Inverse RL and adversarial imitation

Inverse reinforcement learning infers the reward function that would make the expert's behaviour optimal; the agent then optimises that reward and can generalise to new situations. **GAIL** trains a discriminator to tell expert from agent behaviour and uses it as a reward. Both are useful when the right reward is hard to write down — "fly like an experienced wingman" — but they inherit any biases or bad habits in the demonstrations.

23.4 Combining demonstrations with RL

A common recipe: pre-train the policy by behavioural cloning, then fine-tune with PPO or SAC, optionally keeping demonstrations in the replay buffer (as in DQfD) or adding a penalty for straying far from expert behaviour early in training. The agent starts competent and then improves beyond the demonstrator where the reward allows.

24 Hierarchical Control

Military tasks have natural levels: a mission is a sequence of phases, a phase a sequence of manoeuvres, a manoeuvre a sequence of control inputs. **Hierarchical RL** mirrors this. In the **options** framework an option is a temporally extended action — "fly to waypoint", "orbit", "evade" — with its own policy and termination condition; a high-level policy chooses options every few seconds while low-level policies issue thrust commands every few milliseconds. Benefits: exploration happens at a meaningful scale, low-level skills are reused across missions, and the high-level decisions can be inspected by humans. In practice the low level is often a classical controller or a separately trained policy, and only the high level is learned with RL.

25 Learned Control Versus Classical Control

25.1 What classical control already does well

PID controllers, the linear–quadratic regulator (LQR) and model-predictive control (MPC) fly almost every aircraft and missile today. LQR computes the optimal linear feedback for linear dynamics and quadratic costs in closed form; MPC solves a finite-horizon optimisation at every step using a model and re-plans as new measurements arrive, handling constraints explicitly. These methods come with stability proofs and well-understood failure modes — properties that certification authorities require.

25.2 Where RL adds value

RL is attractive where models are poor or decisions are combinatorial: aggressive manoeuvres near the edge of the envelope, contact-rich manipulation, multi-vehicle coordination, and tactical decision layers above the flight controller. A frequent and sensible architecture is **RL on top of classical control**: the learned policy issues set-points or manoeuvre choices; a proven inner-loop controller turns them into actuator commands.

25.3 Model-based RL

Learning a dynamics model from data and then planning with it (MPC on a learned model, or Dyna-style training on imagined rollouts) can cut the number of real interactions by orders of magnitude. The risk is **model exploitation**: the optimiser finds actions that look excellent in the imperfect model and fail in reality. Ensembles of models, whose disagreement signals uncertainty, and short planning horizons mitigate it.

26 Crossing the Reality Gap

- **System identification:** measure the real platform (mass, inertia, motor curves, latency) and calibrate the simulator before training.
- **Domain randomisation:** train across wide ranges of the parameters that cannot be pinned down (wind, sensor noise, delays, payload), so that reality is just one more sample. In the lecture, field success rose from 78% to 88–91% with randomisation.
- **Robust (adversarial) RL:** train against a learned adversary that applies worst-case disturbances within limits, producing policies that hold up under gusts and actuator faults.
- **Fine-tuning on hardware:** a short period of careful real-world learning, often with a safety controller ready to take over.
- **Measuring the gap:** compare the same metrics in simulation and in matched field trials; a policy whose behaviour (not only success rate) differs markedly between the two is relying on a simulator artefact.

27 Assuring a Learned Controller

A neural policy cannot be inspected rule by rule, yet airworthiness and weapons-review processes demand evidence of safe behaviour. Three complementary approaches are emerging:

- **Runtime assurance (the simplex architecture).** The learned controller flies while a simple, verified monitor checks every command against a safety envelope (altitude floor, geofence, separation). If a command would leave the envelope, a verified backup controller takes over. The learned policy brings performance; the verified backup bounds the risk.

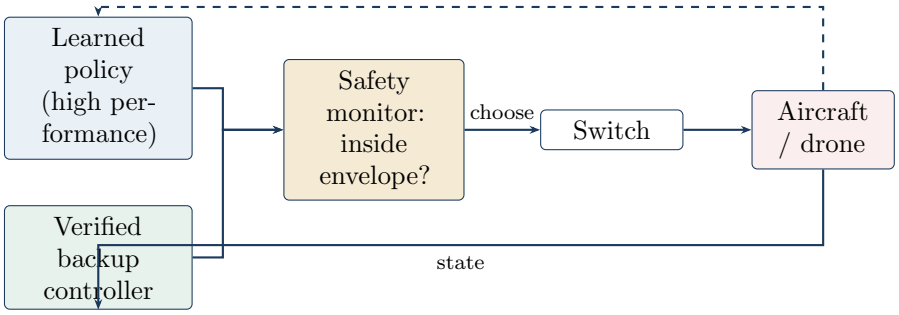


Figure C1: runtime assurance — the learned policy flies unless its command would leave the verified safe envelope.

- **Formal verification of networks.** For small networks, solvers can prove properties such as "for all inputs in this region, the output stays within these bounds". Reachability analysis over the closed loop is computationally heavy but growing.
- **Scenario-based testing.** Systematic and adversarial search for failure cases in simulation — searching specifically for wind profiles, sensor faults or opponent behaviours that break the policy — gives far more assurance than random testing.

28 Designing a Reward for a Drone Swarm

A reward is a weighted sum of terms, and the weights are a statement of priorities. A typical waypoint-following reward per step is

$$r_t = k_p\,(d_{t-1} - d_t) \;-\; k_u\,\|u_t\|^2 \;-\; k_s\,\mathbf{1}[\text{spacing} < s_{\min}] \;-\; k_c\,\mathbf{1}[\text{collision}] \;-\; k_0,$$

rewarding progress toward the waypoint, penalising control effort, unsafe spacing, collisions, and elapsed time.

Worked example — one step's reward

With $k_p = 1$, $k_u = 0.5$, $k_s = 2$, $s_{\min} = 30$ m, $k_0 = 0.01$: the drone closes from 42.0 m to 39.5 m (progress 2.5), uses control effort $\|u\| = 0.6$ (penalty $0.5 \times 0.36 = 0.18$) and is 28 m from a neighbour (spacing penalty 2). Reward $= 2.5 - 0.18 - 2 - 0.01 = 0.31$. The spacing penalty nearly cancels the progress: the agent is being told that keeping formation matters as much as speed. Doubling k_p would tell it the opposite — and it would learn to cut in on its neighbours.

Pitfall — shaping that is not potential-based

The progress term $k_p(d_{t-1} - d_t)$ is potential-based ($\Phi = -k_p d$) and does not change the optimal policy; a term such as "+1 for every step spent within 50 m of the waypoint" is not, and invites the agent to loiter near the waypoint instead of reaching it.

29 Further Self-Test Questions

Check your understanding — Part C

(1) Why do transition probabilities disappear from the policy gradient? (2) In the softmax example, what happens to the preferences if $G = -4$? (3) Why can a single large parameter step be dangerous even if the gradient direction is right? (4) What is the main weakness of behavioural cloning and how does DAgger address it? (5) In a simplex architecture, what happens if the learned policy commands a descent below the altitude floor?

Answers. (1) They do not depend on θ, so their log-derivatives are zero. (2) They move by $-0.4 \times$ score: flank's preference falls by 0.27 and the others rise, making flank less likely. (3) It may change the action distribution drastically (large KL), collapsing performance; trust regions bound the behavioural change. (4) Compounding error in states the expert never visited; DAgger queries the expert on the learner's own states. (5) The monitor rejects the command and the verified backup controller takes over until the aircraft is back inside the envelope.

30 Additional Self-Test Questions

Check your understanding — four more

1. With rewards 2, 0, 0 (terminal), $V = (1, 1, 1, 0)$, $\gamma = 1$, compute $\delta_0, \delta_1, \delta_2$ and the GAE advantage $\hat{A}_0$ for $\lambda = 0.5$. **2.** What happens to PPO if the clip fraction is 0.6? **3.** Why does SAC take the minimum of two critics? **4.** Why must the tanh correction appear in $\log \pi$?

Answers. 1: $\delta_0 = 2 + 1 - 1 = 2$, $\delta_1 = 0 + 1 - 1 = 0$, $\delta_2 = 0 + 0 - 1 = -1$; $\hat{A}_2 = -1$, $\hat{A}_1 = 0 + 0.5(-1) = -0.5$, $\hat{A}_0 = 2 + 0.5(-0.5) = 1.75$. 2: updates are too aggressive (learning rate or epochs too high); training may become unstable. 3: to counter over-estimation bias

from maximising noisy Q estimates. 4: the squashing changes the density; without the Jacobian term probabilities are wrong and the policy saturates at the limits.

31 Across the Disciplines

PID control (Minorsky, 1922) still flies most drones; starling flocks coordinate by tracking about six or seven neighbours (Ballerini et al., 2008) and Reynolds' boids (1987) reproduce flocking from three local rules; Fitts' law (1954) captures the speed–accuracy trade-off in human movement, echoed by the learned σ; OpenAI's Dactyl hand learned manipulation with PPO and heavy domain randomisation before transferring to hardware; the likelihood-ratio gradient estimator was developed for optimising stochastic simulations (Glynn, 1990) and is the same estimator as REINFORCE.

32 Self-Test Questions

Check your understanding — six questions

1. Why can't DQN handle a 12-dimensional continuous thrust action directly? **2.** Show that a constant baseline b does not change the expected gradient. **3.** Compute L^{CLIP} for $r = 1.4$, $\hat{A} = 2$, $\varepsilon = 0.2$. **4.** $\mu = 0$, $\sigma = 1$, $a = -0.5$, $\hat{A} = -2$: in which direction does μ move? **5.** Give two operational benefits of SAC's entropy term. **6.** Why does domain randomisation narrow the sim-to-real gap?

Answers. 1: $\arg\max_a$ over a continuous space is an optimisation at every step. 2: $\mathbb{E}[\nabla \log \pi]\, b = b \nabla \sum \pi = 0$. 3: $\min(2.8,\, 1.2 \times 2 = 2.4) = 2.4$. 4: $\partial \log \pi / \partial \mu = -0.5$; times $\hat{A} = -2$ gives $+1$, so μ increases (away from the bad action). 5: robustness to disturbances (88% vs 71% in wind) and unpredictability to adversaries; also automatic exploration. 6: the policy learns to succeed across a range of dynamics that includes the real one.

33 Glossary

Advantage	$Q(s,a) - V(s)$: how much better an action is than average.
Actor–critic	Policy (actor) trained with a learned value function (critic).
Domain randomisation	Training across randomised simulator parameters for robustness.
Entropy bonus	Reward for keeping the policy stochastic.
GAE	Generalised advantage estimation, trading bias against variance via λ.
On-/off-policy	Learning only from the current policy's data vs from any data.
Probability ratio	$\pi_\theta / \pi_{\theta_{\text{old}}}$ for an action, clipped by PPO.
Score function	$\nabla_\theta \log \pi_\theta(a \mid s)$.